

STEP-BY-STEP PROJECT CONFIGURATION

Microsoft Foundry gpt-sol Configuration Guide

Exactly what to collect from Foundry, where to place each value in BidSight, how to validate the connection, and how to troubleshoot it safely.

Integration	Direct OpenAI-compatible Foundry model call
Authentication	API key stored only in local .env
Deployment	gpt-sol by default; exact deployment name is configurable
Updated	11 August 2026

Decision-support principle

Microsoft Foundry gpt-sol interprets quotation language and explains verified results. Deterministic Python code performs compliance checks, arithmetic, scoring, and ranking. The final purchasing decision remains with an authorised user.

Prepared for the BidSight portfolio project by Badar Butt

1. Before you start

This guide assumes you already have access to Microsoft Foundry and can see a deployed GPT model. BidSight does not provision Azure resources. It only calls your existing deployment.

1.1 You need exactly three Foundry values

Value	Example placeholder	BidSight environment name
Resource endpoint	https://my-resource.services.ai.azure.com	FOUNDRY_ENDPOINT
API key	Never place in documents or source	FOUNDRY_API_KEY
Deployment name	gpt-sol	FOUNDRY_MODEL_DEPLOYMENT

Model name versus deployment name

BidSight sends the deployment name in the model parameter. If the Foundry portal shows the model family as GPT but the deployment is named gpt-sol, enter gpt-sol. Copy the spelling exactly.

1.2 What BidSight does not need

- A Foundry hosted agent, agent ID, vector index, knowledge source, or multi-agent workflow.
- Azure AI Projects SDK, a previous-provider key, Azure OpenAI API version, or a Playground URL.
- Any Foundry value in the frontend. Provider secrets remain backend-only.

2. Confirm the gpt-sol deployment in Foundry

2.1 Open your Foundry resource

- Sign in to Microsoft Foundry using the Azure account that owns or can access the resource.
- Open the resource that contains your GPT deployment.
- Open Models and endpoints or the deployment list, depending on the portal view.
- Confirm the deployment provisioning state is Succeeded or Ready.

2.2 Check the deployment name

Open the deployment details and locate Deployment name. For this repository the recommended value is:

```
gpt-sol
```

If your deployment has a different name, use that exact value instead. Do not rename code or schemas; change only `FOUNDRY_MODEL_DEPLOYMENT`.

2.3 Check model capability

- The deployment must support chat completions and structured outputs.
- The API must be reachable through the resource OpenAI v1 endpoint.
- Your subscription/resource must have usable quota for the deployed model.

Why structured outputs matter

BidSight supplies a Pydantic schema so extraction and recommendation responses can be validated before use. An ordinary free-form chat response is not accepted as application data.

3. Copy the endpoint and key safely

3.1 Resource endpoint

In the deployment or resource details, copy the resource inference endpoint. Supported examples are:

```
https://YOUR-RESOURCE-NAME.services.ai.azure.com  
https://YOUR-RESOURCE-NAME.openai.azure.com/openai/v1/
```

The backend adds /openai/v1/ when the resource-root form is used. Do not paste a browser page URL, a URL containing /projects/, or a URL containing a deployment-specific chat route.

3.2 API key

- Open the Keys and Endpoint area for the same Foundry resource.
- Copy one active key directly into your local .env file.
- Do not send it in chat, commit it, or show it during a demo.

3.3 If you cannot see keys

Your Azure role may not allow key listing, or key access may be disabled. Ask the resource owner for appropriate access. The current BidSight MVP implements key-based authentication only; Microsoft Entra ID can be added later if required.

Secret handling

If a real key is ever committed or appears in a screenshot, rotate it in Foundry immediately and replace the local .env value.

4. Put the values in BidSight

4.1 Create the root .env

At the repository root, copy .env.example to .env. Keep .env untracked. Fill in the Foundry lines as follows:

```
DATABASE_URL=sqlite:///./bidsight.db
FOUNDRY_ENDPOINT=https://YOUR-RESOURCE-NAME.services.ai.azure.com
FOUNDRY_API_KEY=PASTE_REAL_KEY_ONLY_HERE
FOUNDRY_MODEL_DEPLOYMENT=gpt-sol
FOUNDRY_REQUEST_TIMEOUT_SECONDS=90
UPLOAD_DIR=uploads
FRONTEND_URL=http://localhost:3000
MAX_UPLOAD_SIZE_MB=10
```

4.2 Create the frontend env file

Create frontend/.env.local from frontend/.env.example. It contains only the public backend URL:

```
NEXT_PUBLIC_API_URL=http://localhost:8000
```

Never put FOUNDRY_API_KEY in frontend/.env.local

Variables prefixed NEXT_PUBLIC can be exposed to the browser. Foundry endpoint and key are read by FastAPI from the root .env or backend/.env only.

4.3 Configuration precedence

The backend settings module reads the root .env and backend/.env, while real operating-system environment variables may override file values. Keep one intentional local source to avoid confusion.

5. Where Foundry is used in the code

Location	Purpose
backend/app/config.py	Loads FOUNDRY_ENDPOINT, key, deployment, and timeout.
backend/app/services/foundry_service.py	Creates the OpenAI client, prompts gpt-sol, validates responses.
backend/app/schemas.py	Defines FoundryQuotationExtraction and FoundryRecommendation.
backend/app/routers/quotations.py	Calls extraction after PyMuPDF text extraction and stores typed fields.
backend/app/routers/evaluations.py	Sends verified comparison evidence for recommendation.
backend/app/services/scoring_service.py	Calculates all deterministic scores; never calls Foundry.
backend/tests/test_foundry_service.py	Checks config and response validation without a live call.

5.1 Request flow

PDF upload
-> PyMuPDF text
-> Foundry gpt-sol structured extraction
-> Pydantic validation
-> user review
-> Python scoring
-> Foundry gpt-sol recommendation
-> Pydantic validation

5.2 Provider boundary

- Foundry receives quotation text only during extraction.
- Foundry receives verified structured evidence and Python scores during recommendation.
- Foundry never writes directly to SQLite and never calculates scores.
- Routers reject an unknown or ineligible recommended vendor.

6. Refresh dependencies and start the project

6.1 Backend

```
cd backend
uv sync --group dev
uv run pytest
uv run uvicorn app.main:app --reload
```

uv sync installs the OpenAI Python SDK declared by the backend and updates backend/uv.lock.

6.2 Frontend

```
cd frontend
pnpm install
pnpm dev
```

6.3 Open the application

- Backend status: <http://localhost:8000>
- Backend health: <http://localhost:8000/api/health>
- Swagger documentation: <http://localhost:8000/docs>
- Frontend: <http://localhost:3000>

No live Foundry call happens at startup

Health checks confirm the local API, not model credentials. The first real Foundry request happens when a quotation is processed or a recommendation is generated.

7. Validate gpt-sol in BidSight

7.1 Safe smoke test

- Create the sample evaluation using the values in the Run and Demo Guide.
- Upload only techcore-solutions-quotation.pdf first.
- Select the processing action and wait for the extraction review.
- Confirm vendor, model, quantity, prices, delivery, warranty, payment, support, and specifications.
- If this succeeds, upload the other two quotations and continue the full workflow.

7.2 Expected TechCore extraction

Field	Expected value
Vendor	TechCore Solutions
Model	Dell Latitude 5550
Quantity	25
Total price	PKR 3,625,000
Delivery	10 days
Warranty	36 months
RAM / storage	16 GB / 512 GB SSD

7.3 What success proves

- The resource endpoint, API key, and deployment name are correct.
- gpt-sol supports the structured chat-completion response used by BidSight.
- PDF text extraction and Pydantic validation are working end to end.

8. Troubleshooting

Error or symptom	Likely cause and fix
Foundry endpoint is not configured	Set FOUNDRY_ENDPOINT in root .env and restart FastAPI.
Foundry API key is not configured	Set FOUNDRY_API_KEY in root .env; do not add quotes unless part of the key.
401 / authentication failure	Key and endpoint may belong to different resources, or the key was rotated.
404 / deployment not found	FOUNDRY_MODEL_DEPLOYMENT does not exactly match the Foundry deployment name.
429 / quota or rate limit	Wait and retry, or review capacity/quota for the deployment.
Malformed structured data	Confirm gpt-sol supports structured output; retry and inspect provider-side model deployment.
Timeout	Check network/resource availability; increase timeout up to 300 seconds only if necessary.
PDF remains in error	Correct configuration and use Retry; the stored PDF is intentionally preserved.

8.1 Portal URL versus API endpoint

Common mistake

A browser URL from the Foundry portal is not the inference endpoint. Use the resource endpoint from Keys and Endpoint or deployment details.

8.2 Check exact local values without exposing the key

- Confirm the endpoint begins with https:// and contains the correct resource name.
- Confirm deployment is exactly gpt-sol or your actual deployment name.
- Confirm the key line is present and non-empty, but never print or screenshot it.
- Restart FastAPI after changing .env because cached settings are loaded at process start.

9. Security and completion checklist

9.1 Security checklist

- [] Real API key exists only in untracked .env or a secure host secret store.
- [] .env, backend uploads, local databases, node_modules, and .next remain ignored.
- [] No real key appears in Git history, README, PDFs, terminal screenshots, or demo recordings.
- [] The Foundry key is rotated if exposure is suspected.
- [] Provider errors returned to the browser contain no credential or request trace.

9.2 Final working checklist

- [] Foundry deployment state is ready and its exact name is known.
- [] Root .env contains endpoint, key, gpt-sol deployment, and timeout.
- [] uv sync has refreshed backend dependencies and uv.lock.
- [] Backend tests pass without a live provider call.
- [] FastAPI health and Swagger pages open.
- [] Frontend Settings reports Connected.
- [] TechCore PDF reaches editable extraction review through a live gpt-sol call.
- [] Full three-vendor demo recommends TechCore from deterministic evidence.

Done

Once every item above is checked, BidSight is locally configured to use your Microsoft Foundry gpt-sol deployment for extraction and recommendation while Python remains the scoring authority.

Official references

Foundry model endpoints: <https://learn.microsoft.com/azure/foundry/foundry-models/concepts/endpoints>

Foundry structured outputs: <https://learn.microsoft.com/azure/foundry/openai/how-to/structured-outputs>